

Linear algebra (40pts)

- 1.1. Write this fixed point iteration in matrix form

$$x_1 = 0.2x_2 + 0.7x_3$$

$$x_2 = 0.1x_1 + 0.8x_3$$

$$x_3 = 0.2x_1 + 0.7x_2$$

- 1.2. Is the matrix sub-stochastic?
- 1.3. Let $x_1 = 1, x_2 = 1, x_3 = 1$, do 2 iteration by hand using matrix-vector product, and show results

Solution

```
In [3]: # 1.1
import numpy as np

def fix_point(A, x0, tol=1e-6, max_iter=1000):
    x = x0
    for _ in range(max_iter):
        x_new = A @ x
        if np.linalg.norm(x_new - x) < tol:
            return x_new
        x = x_new
    # raise ValueError("Fixed point iteration did not converge within the max_iter")
    return x_new

# 1.2

A = np.array([
    [0.0, 0.2, 0.7],
    [0.1, 0.0, 0.8],
    [0.2, 0.7, 0.0]
])

row_sums = A.sum(axis=1)
is_substochastic = np.all(A >= 0) and np.all(row_sums <= 1)
# the result is [0.9 0.9 0.9], and for each entry it's between 0 and 1, so it is sub-stochastic

# 1.3
x0 = np.array([1.0, 1.0, 1.0])
fixed_point = fix_point(A, x0, tol=1e-6, max_iter=2)
print("Fixed point:", fixed_point)
# The answer is [0.81 0.81 0.81].
```

Fixed point: [0.81 0.81 0.81]

Linear algebra (40pts)

- 2.1 Write a function that takes a square matrix A , an input vector x , and a vector b , and an integer $maxit$ (default =10) and return the result of $maxit$ iterations of

$$x \leftarrow Ax + b$$

- 2.2 Now take A as the matrix in the previous problem. $x = [1, 2, 3]$, $b = [3, 2, 1]$, and do 10, 100, 1000 iterations. Does the iteration process converge? To what?
- 2.4 Now instead of A , use $1.2 * A$, do 10, 100, 1000 iterations, does the iteration converge?
- 2.5 Relate what you see to the contractive mapping theorem

Solution

```
In [20]: # 2.1
import numpy as np

def iterate(A, x, b, maxit=10):
    x = np.array(x, dtype=float)
    b = np.array(b, dtype=float)
    for _ in range(maxit):
        x = A @ x + b
    return x

# 2.2
A = np.array([
    [0.0, 0.2, 0.7],
    [0.1, 0.0, 0.8],
    [0.2, 0.7, 0.0]
])
x0, b = [1.0, 2.0, 3.0], [3.0, 2.0, 1.0]

result10 = iterate(A, x0, b, 10)
result100 = iterate(A, x0, b, 100)
result1000 = iterate(A, x0, b, 1000)
gap_10_100 = np.linalg.norm(result100 - result10)
gap_100_1000 = np.linalg.norm(result1000 - result100)

if gap_100_1000 < 1e-3 and gap_100_1000 < gap_10_100:
    print(f"Converge! Gap shrank from {gap_10_100:.4f} to {gap_100_1000:.2e}")
    print(f"Fixed point ≈ {result1000}")

# 2.4
A2 = 1.2 * A
result10 = iterate(A2, x0, b, 10)
result100 = iterate(A2, x0, b, 100)
result1000 = iterate(A2, x0, b, 1000)
# Print find the answer diverged.
gap_10_100 = np.linalg.norm(result100 - result10)
gap_100_1000 = np.linalg.norm(result1000 - result100)
if gap_100_1000 < 1e-3 and gap_100_1000 < gap_10_100:
    print(f"Converge! Gap shrank from {gap_10_100:.4f} to {gap_100_1000:.2e}")
    print(f"Fixed point ≈ {result1000}")
```

```
else:  
    print("Diverge")
```

Converge! Gap shrank from 8.8017 to 6.68e-04
Fixed point $\approx$ [17.88944724 16.93467337 16.4321608]
Diverge

2.5

$|A|_{\infty} = 0.9 < 1$, the Banach fixed-point theorem preserves the unique fixed point since the Cauchy sequence forms. $|1.2A|_{\infty} = 1.08 > 1$, so the Banach fixed-point theorem can't be preserved.

Reading (20 points)

- Read [Welcome to the Era of Experience](#)
- Write a short reflection (a few sentences) agreeing or disagreeing with the authors' thesis and explain your reasoning.

Summary and self-understanding

Fully summarized with human brain in 43 min.

The Era of Human Data

The growth of AI is good, but the knowledge extracted from human data is rapidly approaching a limit. Especially for breaking the boundary AI need more than existing human data.

The Era of Experience

Data that is generated by the agent interacting with its environment, so it's hard to say what's exact experience. With this procedure, we believe superhuman agents can be achieved.

Streams

Human and animals get info from daily experience stream. AI should do so.

Actions and Observations

AI gets info by calling API. However, AI runs code and observes may be a new form of interaction.

Rewards

Human-centric LLMs typically optimise for rewards based on human prejudgement. The agent cannot discover better strategies that are underappreciated by the human rater. Reward highly relies on the human feedback. To form it formally, the method like bi-level optimisation process is one approach.

Planning and Reasoning

Will the era of experience change the way that agents plan and reason? The growth of AI will always be an echo chamber of existing human knowledge. So building a world model maybe a way to solve.

Why now?

The graph shows the RL using is rapid nowadays, one flaw is agents can self-discover knowledge.

Reinforcement Learning Methods

Already have methods like the Dyna algorithm enabled agents to build and learn from models of their world, allowing them to plan and reason about future actions. RLHF improves ai but brings the bottleneck that agents cannot go beyond existing human knowledge. The era of experience presents an opportunity to revisit and improve classic RL concepts for agents sythetic data can form the world to give out the new form of learning.

Consequences

On the positive side, experiential learning will unlock unprecedented capabilities. Con: ai improvements could also lead to job displacement. The env and reward should be improved. And the info from the world is still needed.

Conclusion

AI is improving, but the limitations still exist.

I partially agree with the author, for these are the proper assumptions of AI evolution. But I also doubt that the recognition itself is totally nonlinear, so we can't use the linear exploitation to discuss the AI behavior. Like AI solving the NS equation is out of expectation but just happen.

Optimization (Don't do since we did not cover)

1. Given function $f(x, y) = \ln x + 2 \ln y$, find the gradient $\nabla f(x, y)$
2. Starting from $x = 1, y = 0.5$, do one step of gradient descent with step size $t = 0.1$. What's the new point?
3. Is the function value smaller at the new point?
4. Can you take a large step size, like $t = 10$? Reason?

Since I have already covered, I just do it here.

1. $\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right) = \left(\frac{1}{x}, \frac{2}{y} \right)$, where $x, y > 0$ for the domain of log function.
2. $(x, y) \leftarrow (1, 0.5) - 0.1 * (1, 4) = (0.9, 0.1)$.
3. By the func below, $f(1, 0.5) \approx -1.39, f(0.9, 0.1) \approx -4.71$, so the value gets smaller.
4. You can't. For it may jump out from the minimal or having the issue of OSCILLATION. For this specific problem, the result of gd is $(-9, -39.5)$, out of the domain.

```
In [ ]: from numpy import log
def f(x,y):
    return log(x) + 2 * log(y)
print(f(1,0.5))
print(f(0.9,0.1))
```

```
-1.3862943611198906
-4.710530701645917
```